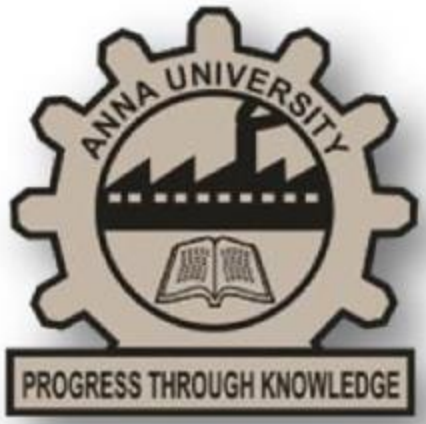
ANNA UNIVERSITY

REGIONAL CAMPUS

COIMBATORE – 641 046



CCS335 – CLOUD COMPUTING

RECORD NOTE BOOK

ANNA UNIVERSITY

REGIONAL CAMPUS

COIMBATORE – 641 046

BONAFIDE CERTIFICATE

Register No | |

This is to certify that this is the Bonafide record of the work done by Mr/Ms _____ in **CCS335 – CLOUD COMPUTING,** ANNA UNIVERSITY REGIONAL CAMPUS COIMBATORE during FIFTH SEMESTER in the academic year 2026-2027.

Staff in Charge **Head of the Department**

Submitted for the practical examination held on _____

Internal Examiner **External Examiner**

TABLE OF CONTENTS

EXP. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.	SIGNATURE
1		Install VirtualBox / VMware / Equivalent open source cloud Workstation with different flavours of Linux or Windows OS on top of Windows 8 and above		
2		Install a C compiler in the virtual machine created using VirtualBox and execute simple programs		
3		Install Google App Engine. Create a hello world app and other simple web applications using Python/Java		
4		Use the GAE Launcher to launch the web applications		
5		Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim		
6		Find a procedure to transfer the files from one virtual machine to another virtual machine		
7		Install Hadoop single node cluster and run simple applications like word count		
8		Creating and Executing Your First Container Using Docker		
9		Run a Container from Docker Hub		

OUTPUT:



Fig 1.1 – VirtualBox Setup – Welcome

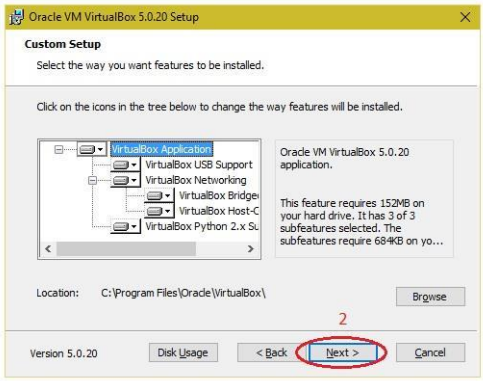


Fig 1.2 – Custom Setup

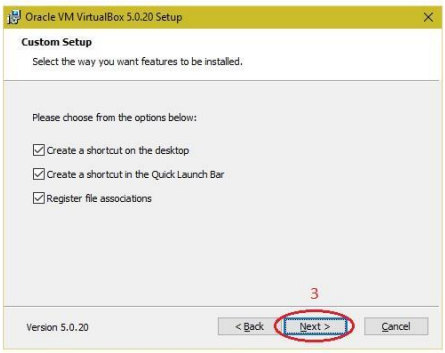


Fig 1.3 – Shortcut options



Fig 1.4 – Network Interfaces warning

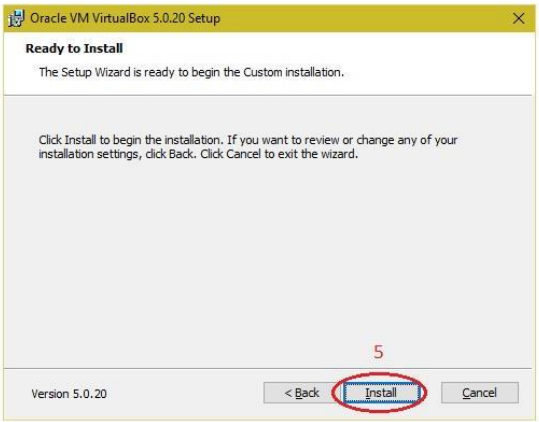


Fig 1.5 – Ready to Install

EX NO. : 1 DATE :	**Install VirtualBox / VMware / Equivalent open source cloud Workstation with different flavours of Linux or Windows OS on top of Windows 8 and above**

AIM:

To install VirtualBox / VMware Workstation (or an equivalent open source cloud platform) on a Windows 8 (or above) host and to create virtual machines running different flavours of Linux and Windows.

REQUIREMENTS:

- **Host:** Windows 8 / 10 / 11 (64-bit), at least 8 GB RAM, 60 GB free disk space, hardware virtualization (Intel VT-x / AMD-V) enabled in BIOS/UEFI.
- **Software:** Oracle VirtualBox (latest version from virtualbox.org), Ubuntu Desktop ISO, a second Linux ISO (e.g. Debian / Fedora / CentOS Stream), Windows 10 ISO (evaluation or licensed).

PROCEDURE:

Part A – Install VirtualBox on Windows

1. Restart the PC, open the BIOS/UEFI setup and enable **Intel VT-x / AMD-V (SVM)**. After booting, check Task Manager → Performance → CPU → Virtualization: Enabled.
2. Download the **Windows hosts** installer from https://www.virtualbox.org/wiki/Downloads and double-click the `.exe` file. On the Welcome screen click **Next**.
3. On **Custom Setup** keep VirtualBox Application, USB Support and Networking selected and click **Next**. Keep the shortcut options ticked and click **Next**.
4. The warning Network Interfaces says the network will be disconnected for a moment. Click **Yes**. If a Python / win32api dependency warning appears, it is optional – click **Yes** to continue.
5. Click **Install** (allow the Windows UAC prompt and any driver-install prompt) and then **Finish**. The VirtualBox Manager window opens.

Part B – Create a Linux VM (Ubuntu)

1. Download the Ubuntu Desktop ISO from https://ubuntu.com/download/desktop.
2. In VirtualBox Manager click **New**. Enter Name: `Ubuntu-VM`, select the downloaded ISO image, Type: **Linux**, Version: **Ubuntu (64-bit)**.
3. Hardware: Base Memory **2048 MB** (4096 MB if available) and **2 CPUs**.
4. Virtual Hard Disk: Create a Virtual Hard Disk Now, size **25 GB**, type VDI, dynamically allocated. Click **Finish**.

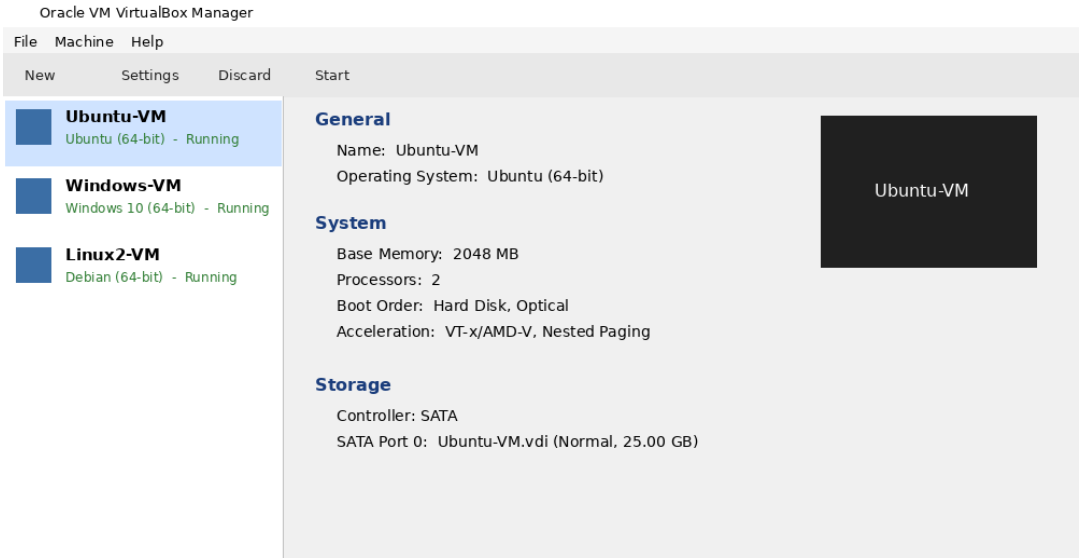


Fig 1.6 – VirtualBox Manager showing the Ubuntu, Windows and Linux virtual machines

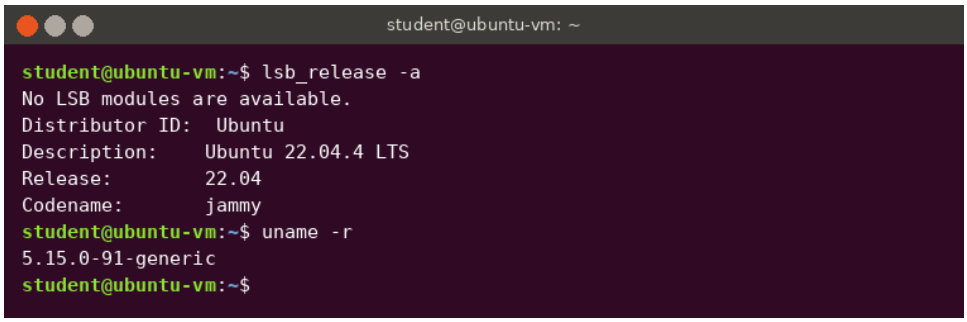


Fig 1.7 – Ubuntu VM – lsb_release -a and uname -r

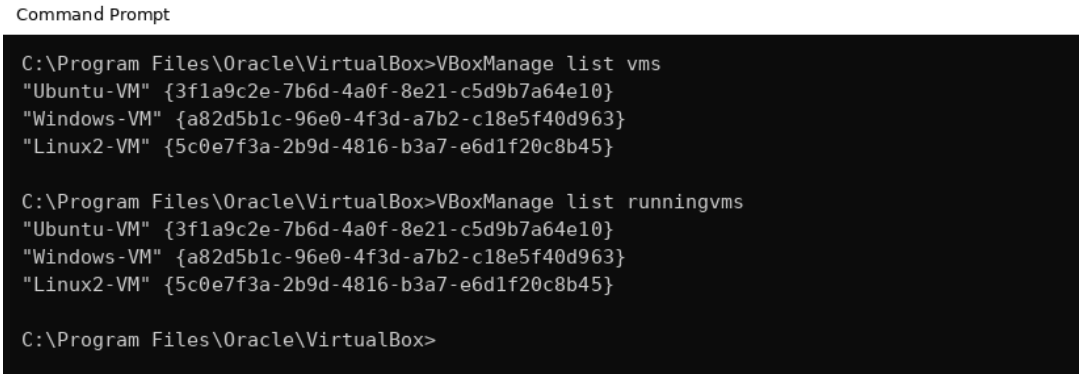


Fig 1.8 – VBoxManage list vms / list runningvms

5. Select the VM and click **Start**. In the installer choose Install Ubuntu, set the user name and password, and wait for the installation to complete. Restart when asked (remove the ISO if prompted).
6. Log in and verify the OS in a terminal:

```
lsb_release -a
uname -r
```

Part C – Create a Windows VM

1. Click **New**. Name: `Windows-VM`, select the Windows 10 ISO, Type: **Microsoft Windows**, Version: **Windows 10 (64-bit)**.
2. Hardware: Base Memory **4096 MB**, **2 CPUs**. Virtual Hard Disk: **50 GB**, VDI, dynamically allocated.
3. Start the VM and complete the Windows installation (choose Custom: Install Windows only and select the unallocated disk).

Part D – Create another Linux flavour

1. Repeat Part B with a different distribution (for example Debian or Fedora). Name it `Linux2-VM`, Type: **Linux**, Version: the matching entry (e.g. Debian (64-bit)).
2. Verify all virtual machines from the Windows command prompt:

```
cd "C:\Program Files\Oracle\VirtualBox"
VBoxManage list vms
VBoxManage list runningvms
```

Part E – Using VMware Workstation (alternative)

1. Install VMware Workstation (Player / Pro) on the Windows host using its installer wizard (Next → Accept licence → Install → Finish).
2. Choose **Create a New Virtual Machine** → Typical → Installer disc image file (iso) → select the guest OS → give a VM name and location → set disk size → **Finish** → **Power on this virtual machine**.

Part F – Equivalent open source cloud platform: OpenNebula Sandbox (optional)

1. Open VirtualBox → **File** → **Import Appliance** → browse to the `OpenNebula-Sandbox .ova` file → **Import**.
2. Start the imported VM. At the console login prompt use user name `root` and the sandbox default password given with the appliance.
3. On the host open a browser and go to `http://localhost:9869` (Sunstone). Log in as `oneadmin` with the sandbox password.
4. Go to **Instances** → **VMs** → **+**, select a template, enter the VM name, number of instances and CPU, then click **Create**. Repeat to create more than one VM.

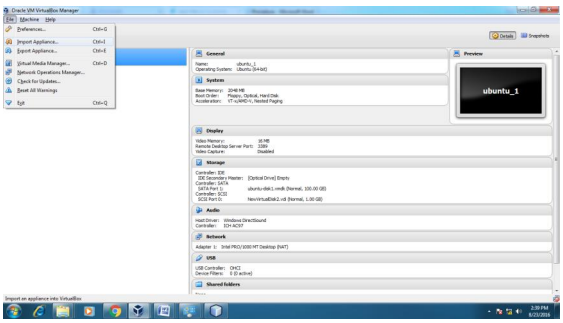


Fig 1.9 – VirtualBox – File → Import Appliance

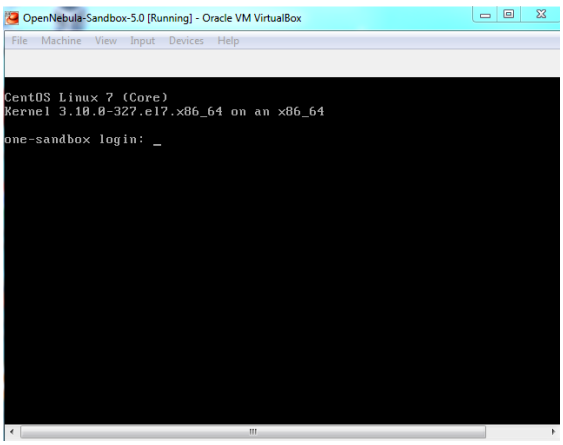


Fig 1.10 – OpenNebula sandbox VM running

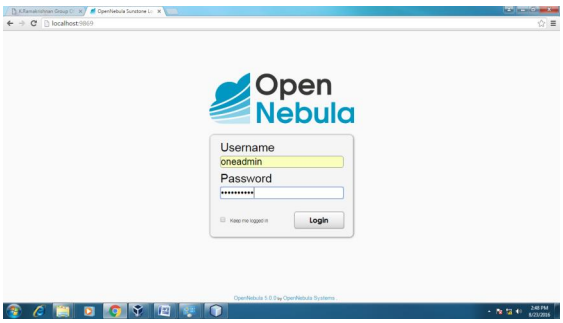


Fig 1.11 – OpenNebula Sunstone login

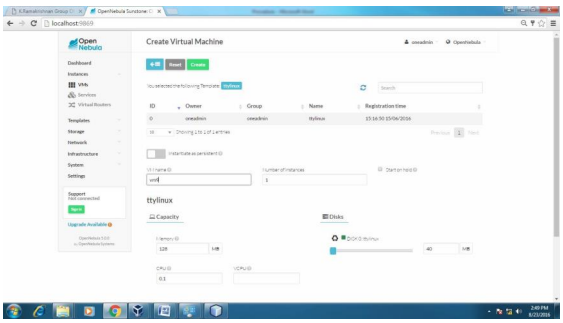


Fig 1.12 – OpenNebula – Create Virtual Machine

VM Name	Guest OS	RAM	Disk	Status
Ubuntu-VM	Ubuntu Linux (64-bit)	2048 MB	25 GB	Running
Windows-VM	Windows 10 (64-bit)	4096 MB	50 GB	Running
Linux2-VM	Debian (64-bit)	2048 MB	25 GB	Running

Virtual machines created

APPLICATIONS:

Virtualization is the foundation of IaaS cloud computing. It lets one physical machine host several isolated operating systems, which is used for server consolidation, software testing on different operating systems, development environments and building private clouds. Large cloud services such as Amazon Web Services run their workloads on virtual machines in the same way.

RESULT:

Thus VirtualBox was installed on a Windows host and virtual machines with different flavours of Linux and Windows were created and executed successfully.

EX NO. : 2 DATE :	**Install a C compiler in the virtual machine created using VirtualBox and execute simple programs**

AIM:

To install a C compiler (GCC) in the Ubuntu virtual machine created using VirtualBox and to execute simple C programs.

REQUIREMENTS:

- Ubuntu-VM created in Experiment 1 (or an imported `.ova` Ubuntu image) with Internet access (default NAT adapter).

PROCEDURE:

1. Start **Ubuntu-VM** from VirtualBox Manager and log in.
2. Open the terminal (`Ctrl+Alt+T`) and update the package list:

```
sudo apt update
```

3. Install the C compiler (the `build-essential` package contains gcc, g++ and make):

```
sudo apt install build-essential -y
```

4. Verify the installation:

```
gcc --version
```

5. Create a working folder and open an editor:

```
mkdir ~/cprograms
cd ~/cprograms
gedit hello.c      # or: nano hello.c
```

6. Type the programs below, save them, then compile with `gcc <file>.c -o <name>` and run with `./<name>`.

Program 1 – hello.c

```
#include <stdio.h>

int main(void)
{
    printf("Hello, Cloud Computing Lab!\n");
    return 0;
}
```

```
gcc hello.c -o hello
./hello
```

Program 2 – add.c (addition of two numbers)

```
#include <stdio.h>

int main(void)
{
```

OUTPUT:

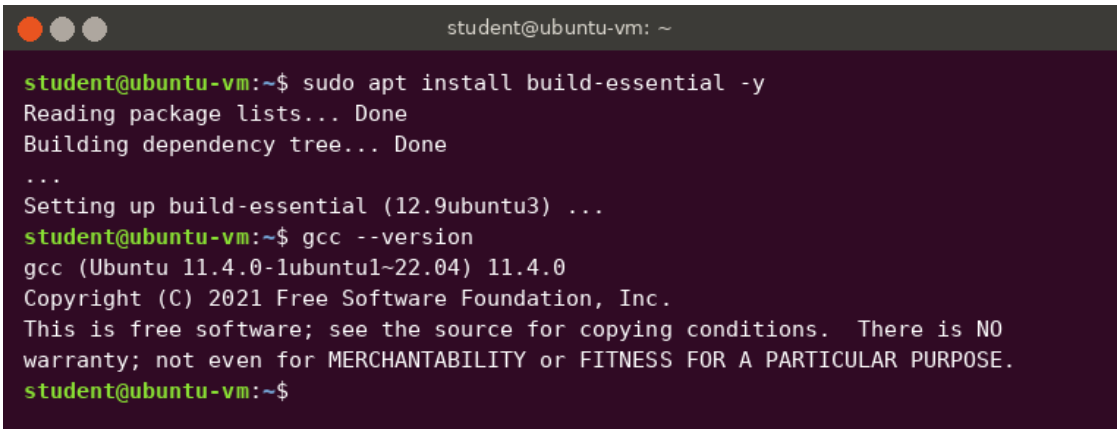


Fig 2.1 – Installing GCC and checking its version

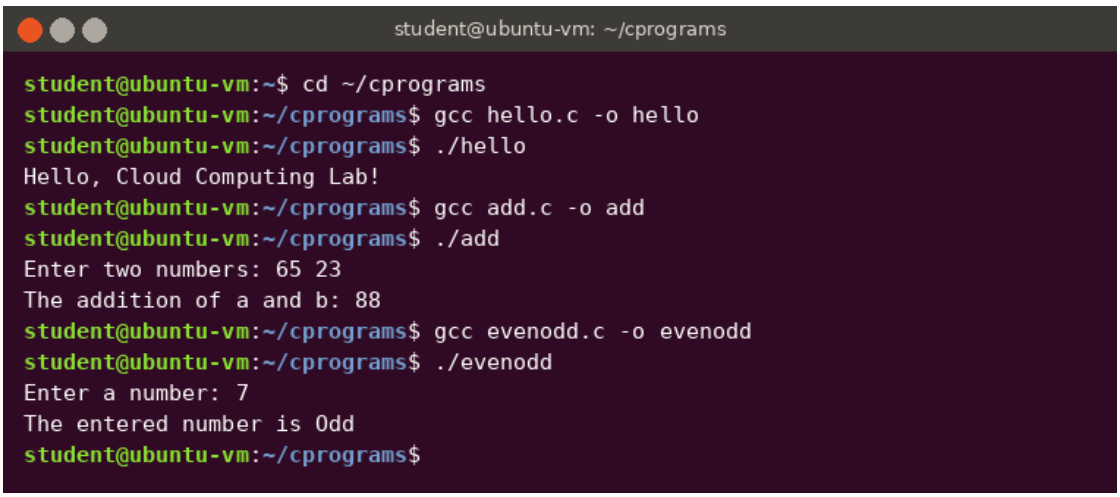


Fig 2.2 – Compilation and execution of the C programs

```
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("The addition of a and b: %d\n", a + b);
    return 0;
}
```

```
gcc add.c -o add
./add
```

Program 3 – evenodd.c (even or odd)

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("The entered number is Even\n");
    else
        printf("The entered number is Odd\n");
    return 0;
}
```

```
gcc evenodd.c -o evenodd
./evenodd
```

APPLICATIONS:

A compiler installed inside a virtual machine gives an isolated, disposable development environment. Programs can be built and tested in the VM without affecting the host, and the same VM image can be copied to other machines or to the cloud.

RESULT:

Thus the C compiler (GCC) was installed in the virtual machine and simple C programs were compiled and executed successfully.

EX NO. : 3 DATE :	**Install Google App Engine. Create a hello world app and other simple web applications using Python/Java**

AIM:

To install Google App Engine (Google Cloud CLI), create a hello world application and other simple web applications using Python, run them locally and deploy them to Google App Engine.

REQUIREMENTS:

- Google account and a Google Cloud project (App Engine deployment requires billing to be enabled on the project; the free quota is sufficient for this lab).
- Python 3.9 or later (3.12 used here), Google Cloud CLI, a text editor.

PROCEDURE:

Part A – Install the Google Cloud CLI

1. Download the Windows installer from https://cloud.google.com/sdk/docs/install, run `GoogleCloudSDKInstaller.exe` and accept the defaults. Tick Run gcloud init at the end.
2. In the **Google Cloud SDK Shell** run `gcloud init`, sign in with your Google account and select (or create) a project.
3. Install the App Engine component and verify:

```
gcloud components install app-engine-python
gcloud --version
```

Part B – Create the Hello World application

1. Create a folder `hello-gae` and, inside it, the three files below.
2. **main.py** – hello world plus a few simple web pages:

```
from datetime import datetime
from flask import Flask
from markupsafe import escape

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello, World! This is my first Google App Engine app."

@app.route("/hello/<name>")
def greet(name):
    return "<h1>Hello, " + str(escape(name)) + "!</h1>"

@app.route("/time")
def show_time():
    return "Server time: " + datetime.now().strftime("%d-%m-%Y %H:%M:%S")
```

OUTPUT:

Command Prompt - python main.py

```
(venv) C:\hello-gae>python main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
 * Debugger is active!
127.0.0.1 - - [06/Oct/2026 10:15:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Oct/2026 10:15:40] "GET /hello/Student HTTP/1.1" 200 -
127.0.0.1 - - [06/Oct/2026 10:16:41] "GET /time HTTP/1.1" 200 -
127.0.0.1 - - [06/Oct/2026 10:17:05] "GET /add/10/20 HTTP/1.1" 200 -
```

Fig 3.1 – Hello World app started locally (python main.py)

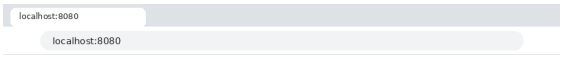


Fig 3.2 – http://localhost:8080/

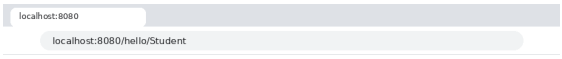


Fig 3.3 – /hello/Student

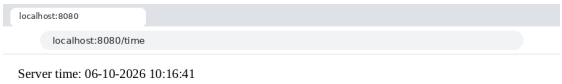


Fig 3.4 – /time

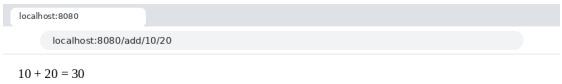


Fig 3.5 – /add/10/20

Google Cloud SDK Shell

```
C:\hello-gae>gcloud app deploy
Services to deploy:

descriptor:                  [C:\hello-gae\app.yaml]
source:                      [C:\hello-gae]
target project:              [cloudlab-gae-2610]
target service:              [default]
target version:              [20261006t102210]
target url:                  [https://cloudlab-gae-2610.el.r.appspot.com]

Do you want to continue (Y/n)?  Y

Beginning deployment of service [default]...
Uploading 3 files to Google Cloud Storage
File upload done.
Updating service [default]...done.
Setting traffic split for service [default]...done.
Deployed service [default] to [https://cloudlab-gae-2610.el.r.appspot.com]
```

Fig 3.6 – gcloud app deploy

```
@app.route("/add/<int:a>/<int:b>")
def add(a, b):
    return str(a) + " + " + str(b) + " = " + str(a + b)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8080, debug=True)
```

3. **requirements.txt:**

```
Flask>=3.0
```

4. **app.yaml** (the configuration file App Engine needs to run and deploy the app):

```
runtime: python312
```

Note: *If python312 is not accepted, use any runtime currently listed in the App Engine Python runtime documentation.*

Part C – Run the application locally

1. Open a command prompt in the `hello-gae` folder and run:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

2. Open `http://localhost:8080/` in the browser. Also test `/hello/<your name>`, `/time` and `/add/10/20`.

Part D – Deploy to Google App Engine

1. Create the App Engine application once per project (choose a region near you, e.g. `asia-south1`):

```
gcloud app create --region=asia-south1
```

2. Deploy from the `hello-gae` folder and answer `Y` when asked:

```
gcloud app deploy
```

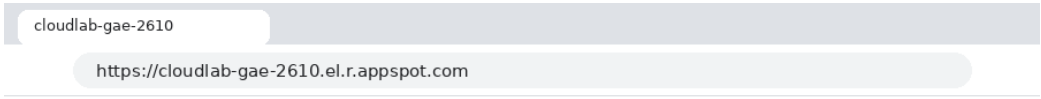
3. Open the deployed application:

```
gcloud app browse
```

The app is served at `https://<PROJECT_ID>.<REGION_ID>.r.appspot.com`.

Java option

The same hello world can be written as a Java servlet web application and deployed with the Google Cloud Tools for Eclipse / App Engine Maven plugin. A standard Java web project contains `src/` (Java sources), `war/WEB-INF/` (web.xml, appengine-web.xml and libraries). The Python procedure above is the one executed in this record.



Hello, World! This is my first Google App Engine app.

Fig 3.7 – Application running on Google App Engine

APPLICATIONS:

Google App Engine is a Platform-as-a-Service (PaaS). Developers upload only the code; the platform handles servers, scaling and load balancing automatically. It is used for web back-ends, REST APIs and mobile application services.

RESULT:

Thus Google App Engine was installed, and a hello world application and other simple web applications were created, run locally and deployed successfully.

OUTPUT:

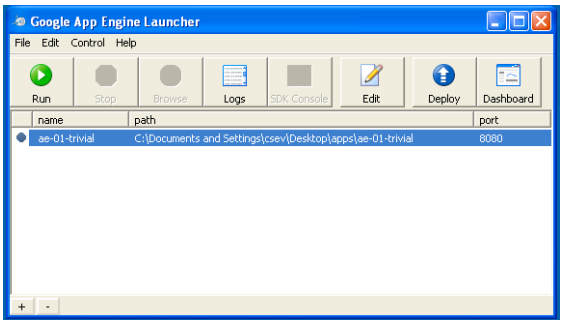


Fig 4.1 – GAE Launcher – application added



Fig 4.2 – Output in the browser (localhost:8080)

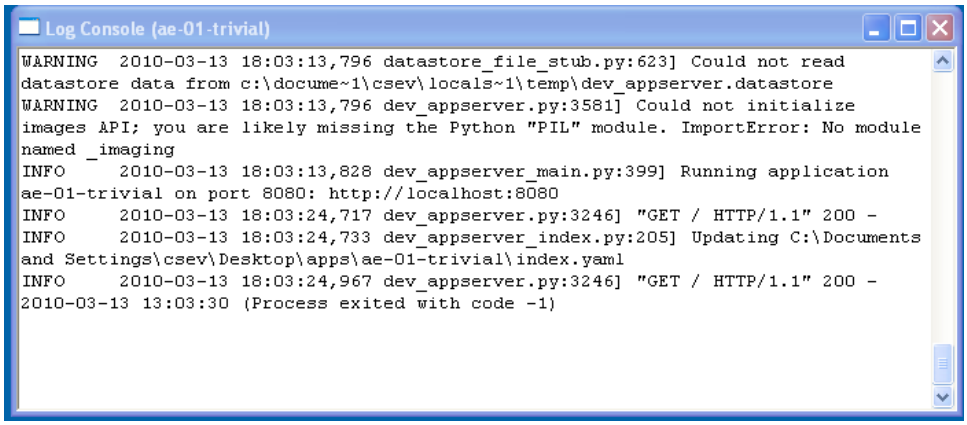


Fig 4.3 – Log console of the application

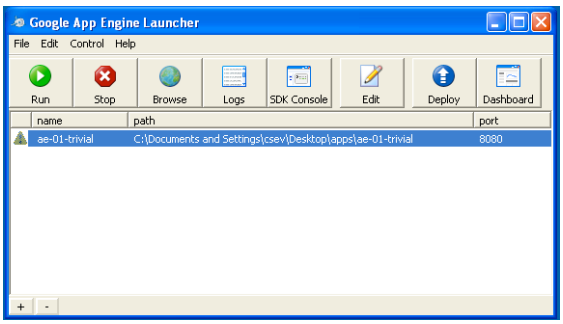


Fig 4.4 – Error in app.yaml – warning icon

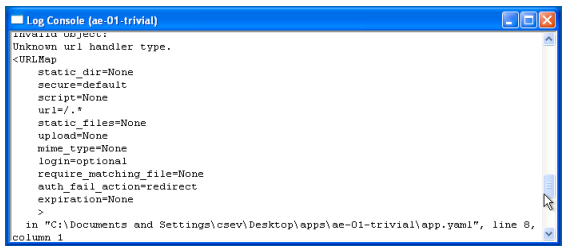


Fig 4.5 – Log showing the app.yaml error

EX NO. : 4 DATE :	Use the GAE Launcher to launch the web applications

AIM:

To use the Google App Engine Launcher to create, run, monitor and stop a web application.

REQUIREMENTS:

- Python 2.7 and the Google App Engine SDK for Python 2.7 (legacy SDK) which contains the **GoogleAppEngineLauncher**.

Note: *The Launcher belongs to the legacy Python 2.7 SDK, so the application in this experiment uses the python27 runtime and Python 2 syntax. If the Launcher is not available, the same application can be started from the SDK command line with* `dev_appserver.py ae-01-trivial`*.*

PROCEDURE:

Part A – Make your first application

1. Create a folder for your App Engine applications, for example `C:\Users\<you>\Desktop\apps`.
2. Inside it create a sub-folder `ae-01-trivial`.
3. In a text editor create **app.yaml** in that folder. Type it by hand (copy-paste may introduce strange characters) and keep every key on its own line with correct indentation:

```
application: ae-01-trivial
version: 1
runtime: python27
api_version: 1
threadsafe: false

handlers:
- url: /.*
  script: index.py
```

4. Create **index.py** in the same folder:

```
print 'Content-Type: text/plain'
print ''
print 'Hello there Chuck'
```

Part B – Launch the application with the GAE Launcher

1. Start **GoogleAppEngineLauncher** from the Start menu / Applications.
2. Choose **File → Add Existing Application**, browse to the `apps\ae-01-trivial` folder, keep port **8080** and click **Add**.

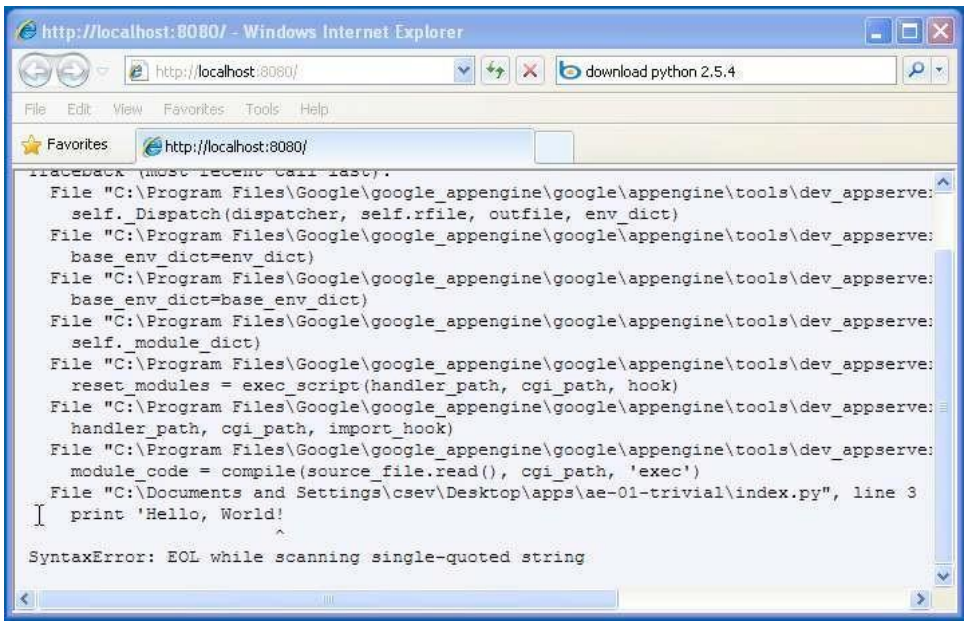


Fig 4.6 – Python traceback in the browser (error in index.py)

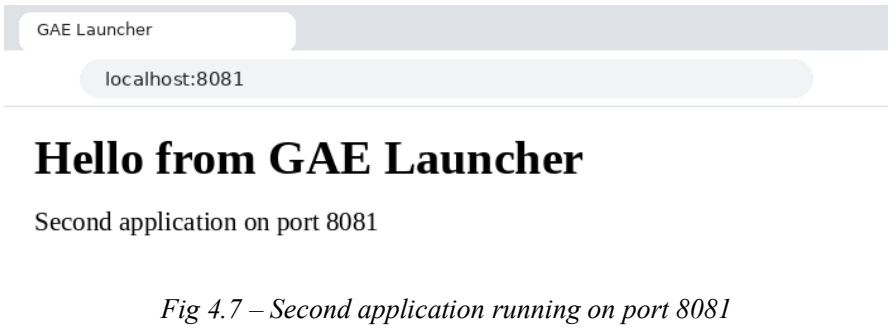


Fig 4.7 – Second application running on port 8081

3. Select the application and click **Run**. After a few moments a green icon appears next to it.
4. Click **Browse** (or open `http://localhost:8080/`) to see the output.
5. Edit `index.py`, change the name "Chuck" to your own name, save and press **Refresh** in the browser to see the update (no restart is needed).

Part C – A second application (HTML output)

1. Create folder `ae-02-html` with an `app.yaml` identical to the one above except `application: ae-02-html`, and this **index.py**:

```
print 'Content-Type: text/html'
print ''
print '<html><head><title>GAE Launcher</title></head>'
print '<body><h1>Hello from GAE Launcher</h1>'
print '<p>Second application on port 8081</p></body></html>'
```

2. Add it to the Launcher with port **8081**, click **Run** and **Browse**.

Part D – Watching the log

Select the application and click **Logs**. Each time the browser page is refreshed the log shows a `GET / HTTP/1.1 200` request.

Part E – Dealing with errors

- **Mistake in app.yaml** (for example a mis-indented line): the application does not start and the Launcher shows a **yellow warning icon**. Open the Logs to see the line number, fix the file and click Run again.
- **Mistake in index.py** (for example a Python syntax error): a traceback appears in the browser. Fix the file and refresh the browser – there is no need to restart the server.

Part F – Shutting down the server

Select the application in the Launcher and click **Stop**.

APPLICATIONS:

The Launcher lets developers test an App Engine application on their own computer before uploading it to the cloud, using the same configuration files and runtime as the real service.

RESULT:

Thus the web applications were created and launched using the Google App Engine Launcher.

EX NO. : 5 DATE :	**Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim**

AIM:

To simulate a cloud scenario using CloudSim and to run a scheduling algorithm (Min-Min) that is not present in CloudSim.

REQUIREMENTS:

- JDK 8 or later, Eclipse IDE for Java Developers.
- CloudSim 3.0.3 (`cloudsim-3.0.3.zip`) from the CloudSim GitHub releases page (https://github.com/Cloudslab/cloudsim/releases). The zip contains the `jars` folder with `cloudsim-3.0.3.jar` and `commons-math3` jar.

THEORY:

CloudSim is a Java library; it need not be installed – the jar files are simply added to the project class path. A simulation uses these entities: **Datacenter** (resource provider made of hosts), **Host** (physical machine with PEs, RAM, bandwidth), **Vm**, **Cloudlet** (a task with a length in million instructions) and **DatacenterBroker** (submits VMs and cloudlets on behalf of the user).

CloudSim provides only simple policies: the broker maps cloudlets to VMs in round-robin order, and a VM executes its cloudlets with the **SpaceShared** or **TimeShared** scheduler. Heuristics such as **Min-Min** are not available, so they have to be written by the user.

PROCEDURE:

Part A – Set up CloudSim in Eclipse

1. Download `cloudsim-3.0.3.zip` and extract it, for example to `C:\cloudsim-3.0.3`.
2. Open Eclipse and choose **File → New → Java Project**. Name it `CloudSimLab` and click **Finish**.
3. Right-click the project → **Build Path → Configure Build Path → Libraries → Add External JARs**. Add `cloudsim-3.0.3.jar` and the `commons-math3` jar from the `jars` folder.
4. To test the set-up, create the package `org.cloudbus.cloudsim.examples` and copy `CloudSimExample1.java` from the extracted `examples` folder into it. Run it as a Java Application.

Part B – Basic steps of a CloudSim simulation

1. Initialise the library: `CloudSim.init(num_user, calendar, trace_flag);`

2. Create the broker: `DatacenterBroker broker = new DatacenterBroker("Broker_0");`
3. Create the VMs (id, user id, MIPS, number of PEs, RAM, bandwidth, image size, VMM, cloudlet scheduler) and submit them: `broker.submitVmList(vmList);`
4. Create the cloudlets (id, length, PEs, file size, output size, utilization models) and submit them: `broker.submitCloudletList(cloudletList);`
5. Start and stop the simulation: `CloudSim.startSimulation();` and `CloudSim.stopSimulation();` then print the results.

Part C – Min-Min scheduling algorithm (not present in CloudSim)

Algorithm

1. Let U be the set of unscheduled cloudlets and `ready[j]` = 0 for every VM j.
2. For every cloudlet i in U and every VM j compute the completion time `CT(i,j) = ready[j] + length(i) / (MIPS(j) x PEs(i))`.
3. For each cloudlet find the VM that gives its minimum completion time, then select the cloudlet whose minimum completion time is the smallest of all.
4. Assign that cloudlet to that VM, set `ready[j] = CT` and remove the cloudlet from U.
5. Repeat steps 2 to 4 until U is empty.

Scenario: 1 datacenter with 2 hosts (4 PEs of 1000 MIPS each), 4 VMs of 250, 500, 750 and 1000 MIPS, and 10 cloudlets of length 4000, 8000, 12000, 16000, 20000, 6000, 10000, 14000, 18000 and 2000 MI.

Program – MinMinScheduling.java (create it in the package `org.cloudbus.cloudsim.examples`):

```
package org.cloudbus.cloudsim.examples;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;

import org.cloudbus.cloudsim.Cloudlet;
import org.cloudbus.cloudsim.CloudletSchedulerSpaceShared;
import org.cloudbus.cloudsim.Datacenter;
import org.cloudbus.cloudsim.DatacenterBroker;
import org.cloudbus.cloudsim.DatacenterCharacteristics;
import org.cloudbus.cloudsim.Host;
import org.cloudbus.cloudsim.Log;
import org.cloudbus.cloudsim.Pe;
import org.cloudbus.cloudsim.Storage;
import org.cloudbus.cloudsim.UtilizationModel;
import org.cloudbus.cloudsim.UtilizationModelFull;
import org.cloudbus.cloudsim.Vm;
import org.cloudbus.cloudsim.VmAllocationPolicySimple;
```

```
import org.cloudbus.cloudsim.VmSchedulerTimeShared;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;

public class MinMinScheduling {

    public static void main(String[] args) {
        Log.printLine("Starting MinMinScheduling...");
        try {
            CloudSim.init(1, Calendar.getInstance(), false);

            createDatacenter("Datacenter_0");
            DatacenterBroker broker = new DatacenterBroker("Broker_0");
            int brokerId = broker.getId();

            List<Vm> vmList = createVms(brokerId);
            List<Cloudlet> cloudletList = createCloudlets(brokerId);

            // Min-Min fixes the VM of every cloudlet and the order
            List<Cloudlet> ordered = minMin(vmList, cloudletList);

            broker.submitVmList(vmList);
            broker.submitCloudletList(ordered);

            CloudSim.startSimulation();
            List<Cloudlet> result = broker.getCloudletReceivedList();
            CloudSim.stopSimulation();

            printResult(result);
            Log.printLine("MinMinScheduling finished!");
        } catch (Exception e) {
            e.printStackTrace();
            Log.printLine("Unwanted errors happen");
        }
    }

    /* Min-Min: repeatedly choose the (cloudlet, VM) pair having the
       smallest completion time and bind the cloudlet to that VM. */
    private static List<Cloudlet> minMin(List<Vm> vms, List<Cloudlet> tasks) {
        List<Cloudlet> unscheduled = new ArrayList<Cloudlet>(tasks);
        List<Cloudlet> ordered = new ArrayList<Cloudlet>();
        double[] ready = new double[vms.size()];   // VM ready times

        while (!unscheduled.isEmpty()) {
            double bestCT = Double.MAX_VALUE;
            Cloudlet bestTask = null;
            int bestVm = -1;

            for (Cloudlet c : unscheduled) {
                for (int j = 0; j < vms.size(); j++) {
                    double exec = c.getCloudletLength()
                            / (vms.get(j).getMips() * c.getNumberOfPes());
```

OUTPUT:

Console - CloudSimExample1

```
Starting CloudSimExample1...
Initialising...
Starting CloudSim version 3.0
Datacenter_0 is starting...
Broker is starting...
Entities started.
0.0: Broker: Cloud Resource List received with 1 resource(s)
0.0: Broker: Trying to Create VM #0 in Datacenter_0
0.1: Broker: VM #0 has been created in Datacenter #2, Host #0
0.1: Broker: Sending cloudlet 0 to VM #0
400.1: Broker: Cloudlet 0 received
400.1: Broker: All Cloudlets executed. Finishing...
Simulation completed.

========== OUTPUT ==========
Cloudlet ID  STATUS   Data center ID  VM ID  Time  Start Time  Finish Time
    0        SUCCESS        2           0    400     0.1        400.1
CloudSimExample1 finished!
```

Fig 5.1 – Output of the existing example CloudSimExample1

Console - MinMinScheduling

```
Starting MinMinScheduling...
Initialising...
Starting CloudSim version 3.0
Datacenter_0 is starting...
Broker_0 is starting...
Entities started.
... (event log omitted) ...
Simulation completed.

========== OUTPUT ==========
Cloudlet ID  STATUS  VM ID  Time  Start  Finish
    9      SUCCESS    3    2    0.1    2.1
    0      SUCCESS    2    5.33    0.1    5.43
    5      SUCCESS    3    6    2.1    8.1
    1      SUCCESS    1    16    0.1    16.1
    6      SUCCESS    3    10    8.1    18.1
    2      SUCCESS    2    16    5.43    21.43
    7      SUCCESS    3    14    18.1    32.1
    3      SUCCESS    2    21.33    21.43    42.77
    8      SUCCESS    3    18    32.1    50.1
    4      SUCCESS    1    40    16.1    56.1
Makespan = 56.1
MinMinScheduling finished!
```

Fig 5.2 – Output of MinMinScheduling

Expected schedule from the Min-Min algorithm (computed from the algorithm; CloudSim adds about 0.1 s start-up time):

```
                    double ct = ready[j] + exec;
                    if (ct < bestCT) {
                        bestCT = ct;
                        bestTask = c;
                        bestVm = j;
                    }
                }
            }
            bestTask.setVmId(vms.get(bestVm).getId());
            ready[bestVm] = bestCT;
            unscheduled.remove(bestTask);
            ordered.add(bestTask);
        }
        return ordered;
    }

    private static Datacenter createDatacenter(String name) throws Exception {
        List<Host> hostList = new ArrayList<Host>();
        for (int h = 0; h < 2; h++) {
            List<Pe> peList = new ArrayList<Pe>();
            for (int p = 0; p < 4; p++) {
                peList.add(new Pe(p, new PeProvisionerSimple(1000)));
            }
            hostList.add(new Host(h, new RamProvisionerSimple(16384),
                    new BwProvisionerSimple(10000), 1000000, peList,
                    new VmSchedulerTimeShared(peList)));
        }
        DatacenterCharacteristics ch = new DatacenterCharacteristics(
                "x86", "Linux", "Xen", hostList, 10.0, 3.0, 0.05, 0.001, 0.0);
        return new Datacenter(name, ch, new VmAllocationPolicySimple(hostList),
                new LinkedList<Storage>(), 0);
    }

    private static List<Vm> createVms(int brokerId) {
        List<Vm> list = new ArrayList<Vm>();
        double[] mips = {250, 500, 750, 1000};
        for (int i = 0; i < mips.length; i++) {
            list.add(new Vm(i, brokerId, mips[i], 1, 512, 1000, 10000, "Xen",
                    new CloudletSchedulerSpaceShared()));
        }
        return list;
    }

    private static List<Cloudlet> createCloudlets(int brokerId) {
        List<Cloudlet> list = new ArrayList<Cloudlet>();
        long[] length = {4000, 8000, 12000, 16000, 20000,
                         6000, 10000, 14000, 18000, 2000};
        UtilizationModel full = new UtilizationModelFull();
        for (int i = 0; i < length.length; i++) {
            Cloudlet c = new Cloudlet(i, length[i], 1, 300, 300,
                    full, full, full);
            c.setUserId(brokerId);
            list.add(c);
```

Order	Cloudlet	Length (MI)	Assigned VM (MIPS)	Finish time (s)
1	9	2000	VM 3 (1000)	2.00
2	0	4000	VM 2 (750)	5.33
3	5	6000	VM 3 (1000)	8.00
4	1	8000	VM 1 (500)	16.00
5	6	10000	VM 3 (1000)	18.00
6	2	12000	VM 2 (750)	21.33
7	7	14000	VM 3 (1000)	32.00
8	3	16000	VM 2 (750)	42.67
9	8	18000	VM 3 (1000)	50.00
10	4	20000	VM 1 (500)	56.00

Makespan with Min-Min is about **56 s**. With simple round-robin binding (cloudlet i to VM i mod 4) it is about **168 s**, because long cloudlets land on the slowest VM (250 MIPS).

```
        }
        return list;
    }

    private static void printResult(List<Cloudlet> list) {
        DecimalFormat dft = new DecimalFormat("###.##");
        double makespan = 0;
        Log.printLine();
        Log.printLine("========== OUTPUT ==========");
        Log.printLine("Cloudlet ID  STATUS  VM ID  Time  Start  Finish");
        for (Cloudlet c : list) {
            if (c.getCloudletStatus() == Cloudlet.SUCCESS) {
                Log.printLine("    " + c.getCloudletId() + "      SUCCESS    "
                        + c.getVmId() + "    "
                        + dft.format(c.getActualCPUTime()) + "    "
                        + dft.format(c.getExecStartTime()) + "    "
                        + dft.format(c.getFinishTime()));
                makespan = Math.max(makespan, c.getFinishTime());
            }
        }
        Log.printLine("Makespan = " + dft.format(makespan));
    }
}
```

Part D – Run and observe

1. Right-click `MinMinScheduling.java` → **Run As → Java Application** and note the console output.
2. Compare the makespan with the default CloudSim behaviour (cloudlet i submitted to VM i mod 4 without the Min-Min step: remove the `minMin` call and submit `cloudletList` directly).

APPLICATIONS:

Simulation allows cloud resource-management and scheduling policies to be evaluated repeatedly, at no cost and without a real datacenter, before they are used in a production cloud.

RESULT:

Thus a cloud scenario was simulated using CloudSim and the Min-Min scheduling algorithm, which is not present in CloudSim, was implemented and executed successfully.

EX NO. : 6 DATE :	**Find a procedure to transfer the files from one virtual machine to another virtual machine**

AIM:

To find a procedure to transfer files from one virtual machine to another virtual machine.

REQUIREMENTS:

- VirtualBox with two Ubuntu virtual machines: **VM1** (sender) and **VM2** (receiver), for example clones of the VM created in Experiment 1.

AVAILABLE METHODS:

- **Shared clipboard / drag and drop** between host and guests (needs Guest Additions) – suitable for a few lines of text or small files.
- **Shared folder** on the host that is mounted in both VMs – a file placed in it is immediately visible in the other VM.
- **Network copy with SCP/SSH** (client–server) – works like copying between two real computers. This is the method executed below.
- Mounting a directory of one VM in the other using NFS, SSHFS or Samba.

PROCEDURE:

Method 1 – Copy files between VMs using SCP

1. Shut down both VMs. For each VM open **Settings → Network → Adapter 1** and set *Attached to* **Bridged Adapter** (or **NAT Network** with the same network name in both VMs) so that the two VMs can reach each other.
2. Start both VMs. In each one find the IP address:

   ```
   hostname -I
   ```

   Assume VM1 = 192.168.1.10 and VM2 = 192.168.1.11 (use your own addresses).
3. On **VM2** install and start the SSH server, then check that it is running:

   ```
   sudo apt update
   sudo apt install openssh-server -y
   sudo systemctl enable --now ssh
   systemctl status ssh
   ```

4. On **VM1** check the connectivity:

   ```
   ping -c 4 192.168.1.11
   ```

5. On **VM1** create a test file:

   ```
   echo "Hello from VM1" > transfer.txt
   ```

6. Copy the file from VM1 to VM2 (replace `user2` with the user name on VM2). Type `yes` for the host-key question and enter the password of `user2`:

   ```
   scp transfer.txt user2@192.168.1.11:/home/user2/
   ```

OUTPUT:

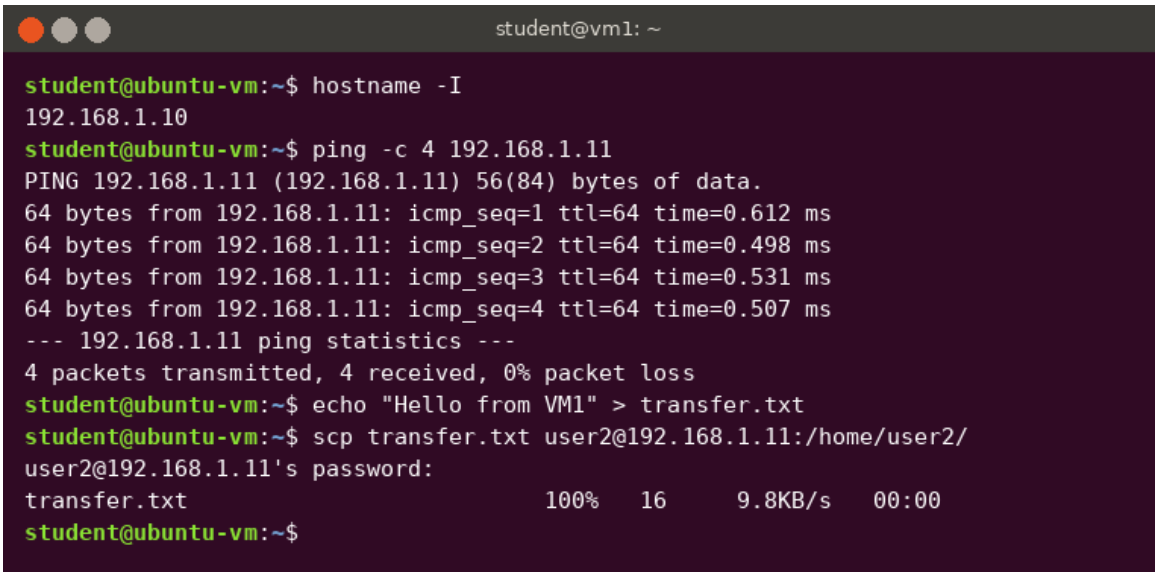


Fig 6.1 – VM1 – IP address, ping and scp command

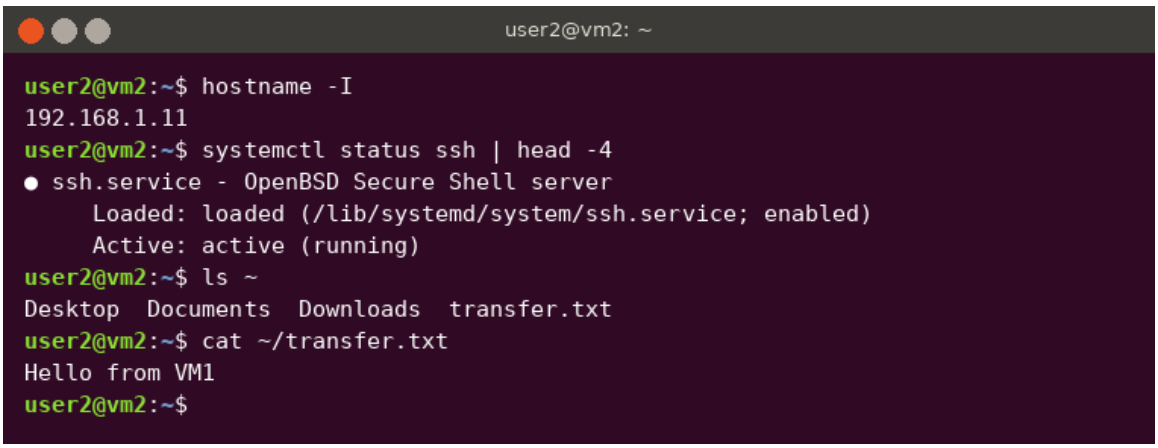


Fig 6.2 – VM2 – SSH service running and the received file

7. On **VM2** verify that the file has arrived:

```
ls ~
cat ~/transfer.txt
```

8. To copy a whole folder, or to fetch a file from VM2 to VM1:

```
scp -r myfolder user2@192.168.1.11:/home/user2/
scp user2@192.168.1.11:/home/user2/transfer.txt .
```

9. (Optional) Avoid typing the password every time by creating an SSH key on VM1 and copying it to VM2:

```
ssh-keygen -t ed25519
ssh-copy-id user2@192.168.1.11
```

Method 2 – Shared folder

1. In each VM choose **Devices → Insert Guest Additions CD image**, run the installer from the mounted CD (`sudo ./VBoxLinuxAdditions.run`) and restart the VM.
2. Create a folder on the host (for example `C:\VMShare`). For each VM open **Settings → Shared Folders → Add**, select the folder and tick **Auto-mount** and **Make Permanent**.
3. In each VM add your user to the `vboxsf` group and restart:

```
sudo adduser $USER vboxsf
sudo reboot
```

4. The folder appears under `/media/sf_VMShare`. A file copied into it from VM1 is immediately visible in VM2.

Method 3 – Shared clipboard and drag and drop

With Guest Additions installed, set **Settings → General → Advanced → Shared Clipboard** and **Drag'n'Drop** to **Bidirectional** in both VMs. Text copied in one VM can then be pasted in the other, and files can be dragged between the VM window and the host.

Note: *Each VM runs as an ordinary program owned by a user of the host operating system, so the host user's folder permissions also apply to shared folders. Clock differences between VMs may show different date/time stamps on shared files.*

APPLICATIONS:

Moving data between virtual machines is required for sharing code and datasets in a virtual cluster, for backups, and for deploying application files to servers running in the cloud.

RESULT:

Thus the procedure to transfer files from one virtual machine to another virtual machine was found and executed successfully.

EX NO. : 7 DATE :	Install Hadoop single node cluster and run simple applications like word count

AIM:

To install a Hadoop single node cluster and run a simple application like word count.

REQUIREMENTS:

- Ubuntu VM (at least 2 GB RAM, 2 CPUs) with Internet access, OpenJDK 8 and Hadoop 2.7.3.

PROCEDURE:

Part A – Install Java and SSH

1. Install OpenJDK 8 and the SSH server and check the Java version:

```
sudo apt update
sudo apt install openjdk-8-jdk ssh -y
java -version
```

2. Hadoop daemons are started through SSH, so create a passwordless key for localhost and test it:

```
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost       # type yes, then: exit
```

Part B – Download and install Hadoop

1. Download and extract Hadoop 2.7.3 in the home directory:

```
cd ~
wget https://archive.apache.org/dist/hadoop/core/hadoop-2.7.3/hadoop-
2.7.3.tar.gz
tar -xzf hadoop-2.7.3.tar.gz
```

2. Add the Java and Hadoop paths at the end of `~/.bashrc` (`vi ~/.bashrc` or `nano ~/.bashrc`):

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop-2.7.3
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export PATH=$PATH:$JAVA_HOME/bin:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

3. Apply the changes to the current terminal and verify both installations:

```
source ~/.bashrc
java -version
hadoop version
```

Part C – Configure Hadoop (files are in hadoop-2.7.3/etc/hadoop)

`cd ~/hadoop-2.7.3/etc/hadoop` and `ls` list the configuration files. Edit them as follows.

1. **hadoop-env.sh** – holds the environment variables used by the Hadoop scripts. Set the Java path:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

2. **core-site.xml** – tells the Hadoop daemons where the NameNode runs. Put this inside the `<configuration>` tag:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

3. **hdfs-site.xml** – settings of the HDFS daemons (replication factor, permissions):

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.permissions.enabled</name>
    <value>false</value>
  </property>
</configuration>
```

4. **mapred-site.xml** – create it from the template and tell MapReduce to use YARN:

```
cp mapred-site.xml.template mapred-site.xml

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

5. **yarn-site.xml** – settings of the ResourceManager and NodeManager:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
    <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
```

OUTPUT:

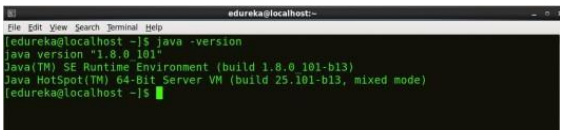


Fig 7.1 – java -version

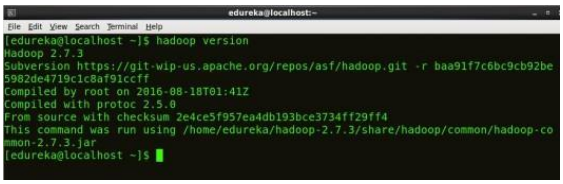


Fig 7.2 – hadoop version

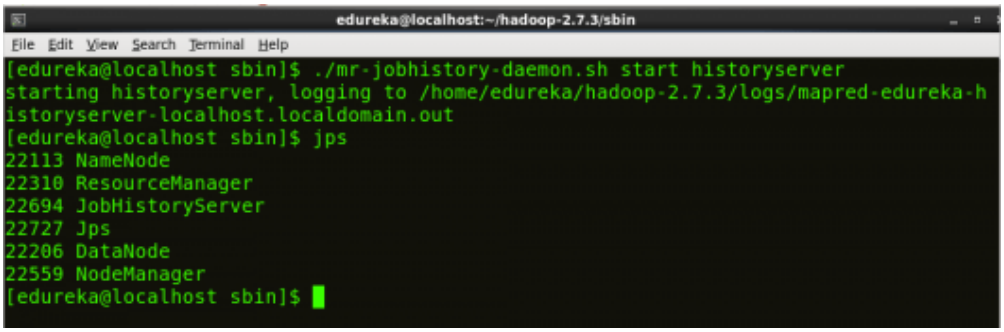


Fig 7.3 – jps – all Hadoop daemons running

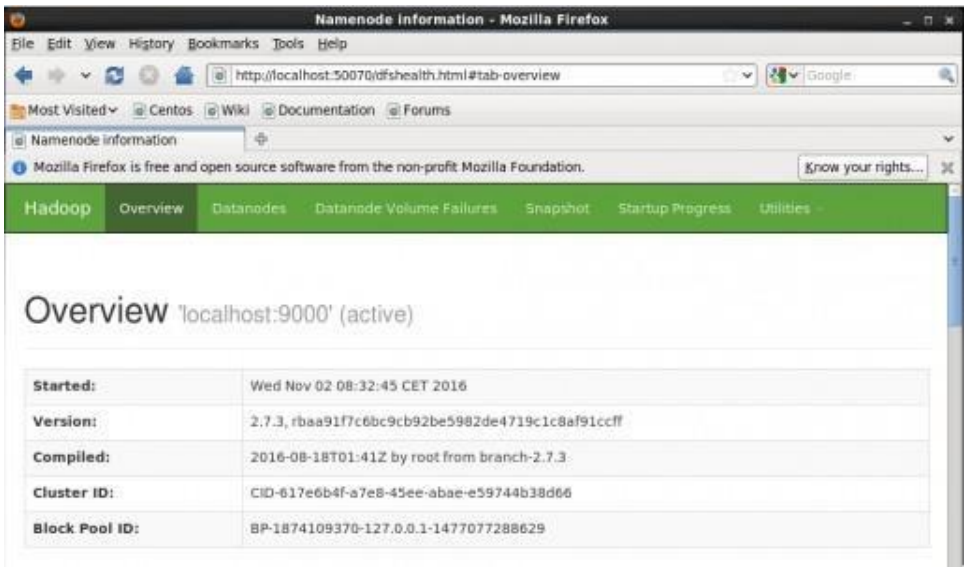


Fig 7.4 – NameNode web interface (localhost:50070)

```
</configuration>
```

Part D – Format the NameNode and start the daemons

1. Format HDFS (only the first time – never format a running file system, you will lose the data stored in HDFS):

```
hdfs namenode -format
```

2. Start HDFS, YARN and the JobHistory server:

```
start-dfs.sh
start-yarn.sh
mr-jobhistory-daemon.sh start historyserver
```

3. Check that all the daemons are running:

```
jps
```

   Expected processes: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, JobHistoryServer and Jps.

4. Open the NameNode web interface in the browser: `http://localhost:50070` (ResourceManager: `http://localhost:8088`).

Note: *Without a `dfs.namenode.name.dir` setting HDFS keeps its data in /tmp, which is cleared on reboot. After a restart of the VM run `hdfs namenode -format` again before starting the daemons.*

Part E – Run the word count application

1. Create a sample input file on the local disk:

```
mkdir ~/wcinput
cat > ~/wcinput/input.txt << EOF
hello hadoop
hello cloud computing
cloud computing with hadoop
EOF
```

2. Create an input directory in HDFS and copy the file into it:

```
hdfs dfs -mkdir -p /user/$USER/input
hdfs dfs -put ~/wcinput/input.txt /user/$USER/input
hdfs dfs -ls /user/$USER/input
```

3. Run the word count MapReduce program supplied with Hadoop (the output directory must not exist already):

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-
2.7.3.jar \
    wordcount /user/$USER/input /user/$USER/output
```

4. List the output directory (it contains `_SUCCESS` and `part-r-00000`) and display the result:

```
hdfs dfs -ls /user/$USER/output
```

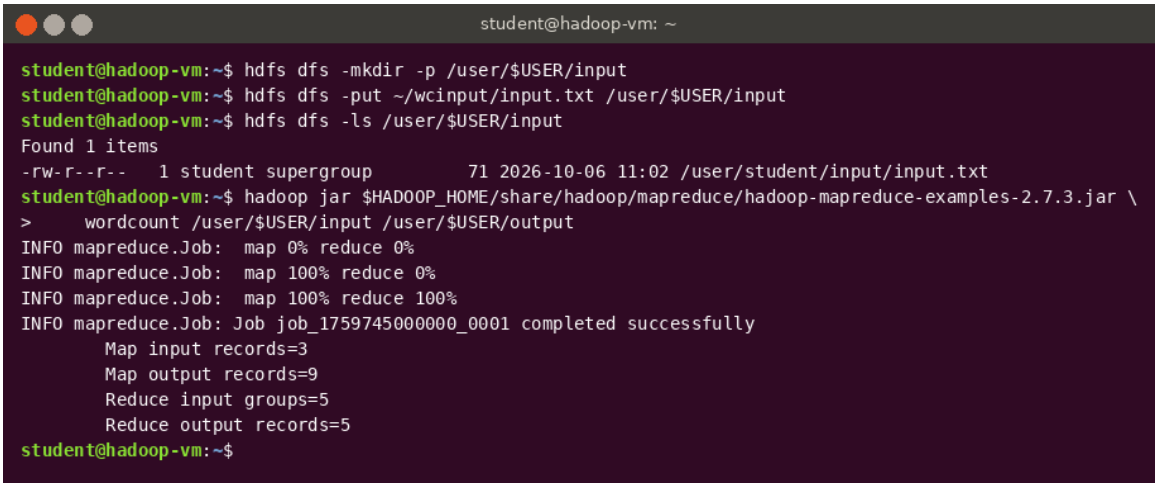


Fig 7.5 – Running the word count MapReduce job

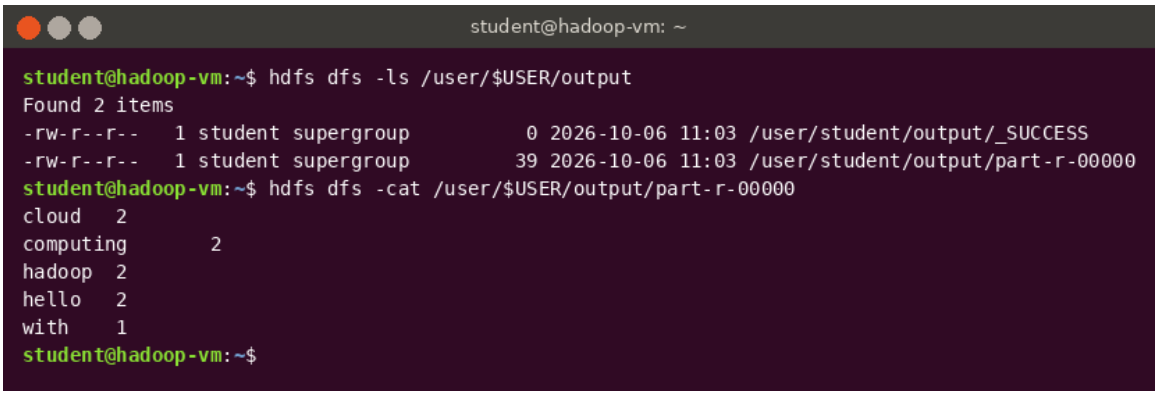


Fig 7.6 – Word count output

```
hdfs dfs -cat /user/$USER/output/part-r-00000
```

Part F – Stop the daemons

```
mr-jobhistory-daemon.sh stop historyserver
stop-yarn.sh
stop-dfs.sh
```

APPLICATIONS:

Hadoop stores very large data sets across a cluster using HDFS and processes them in parallel with MapReduce. Word count is the basic pattern behind log analysis, text mining, search indexing and recommendation systems.

RESULT:

Thus a Hadoop single node cluster was installed and the word count application was executed successfully.

EX NO. : 8 DATE :	Creating and Executing Your First Container Using Docker

AIM:

To create and execute the first container using Docker.

REQUIREMENTS:

- A Docker client: Docker Engine on the Ubuntu VM, Docker Desktop on Windows, or Play with Docker (https://labs.play-with-docker.com).

To install Docker Engine on Ubuntu:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER      # log out and log in again
```

PROCEDURE:

Get started

1. Display the Docker help and the version of the installation:

```
docker --help
docker version
```

`docker version` shows a **Client** and a **Server (Docker Engine)**. Docker has management commands such as container, image, network, volume and system; this lab mainly uses `container`.

Step 1 – Run your first container

1. Open a terminal and run a container from the `ubuntu` image with the `top` command. The `-i` and `-t` flags keep the input open and allocate a pseudo-terminal which `top` needs:

```
docker container run -it ubuntu top
```

Docker first pulls the image (`Unable to find image 'ubuntu:latest' locally ... Status: Downloaded newer image for ubuntu:latest`) and then starts the container. `top` lists only one process – itself – because of PID namespace isolation. The container does not have its own kernel; it uses the host kernel and the image only supplies the Ubuntu file system and tools.

2. Open a **new terminal** and list the running containers:

```
docker container ls
```

3. Use the container ID (or name) to start a bash shell inside the running container:

```
docker container exec -it <CONTAINER ID> bash
root@<id>:/# ps -ef
```

OUTPUT:

Terminal 1

```
$ docker container run -it ubuntu top
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
Digest: sha256:f3a61450ae43896c4332bda5e78b453f4a93179045f20c8181043b26b5e79028
Status: Downloaded newer image for ubuntu:latest

top - 20:32:46 up 3 days, 17:40,  0 users,  load average: 0.00, 0.01, 0.00
Tasks:   1 total,   1 running,   0 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  2046768 total,   173308 free,   117248 used,  1756212 buff/cache

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0   36636   3072   2640 R   0.3  0.2   0:00.04 top
```

Fig 8.1 – First container: docker container run -it ubuntu top

Terminal 2

```
$ docker container ls
CONTAINER ID   IMAGE    COMMAND   CREATED          STATUS          PORTS   NAMES
b3ad2a23fab3   ubuntu   "top"     29 minutes ago   Up 29 minutes           goofy_nobel
$ docker container exec -it b3ad2a23fab3 bash
root@b3ad2a23fab3:/# ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 20:34 ?        00:00:00 top
root        17     0  0 21:06 ?        00:00:00 bash
root        27    17  0 21:14 ?        00:00:00 ps -ef
root@b3ad2a23fab3:/# exit
exit
```

Fig 8.2 – docker container exec – processes inside the container



Fig 8.3 – Nginx server at localhost:8080



Fig 8.4 – MongoDB at localhost:8081

Only `top`, `bash` and `ps` are visible. This is not ssh into another machine – the container is just a group of isolated processes on the host.

4. Exit the container shell and compare with the host:

```
exit
ps -ef            # many processes are shown on the host
```

5. Clean up: in the first terminal press `Ctrl+C` to stop `top`, then list all containers and remove the stopped one:

```
docker ps -a
docker rm <CONTAINER ID>
```

Note: *Linux namespaces give containers their isolation: PID (process IDs), MNT (mount points), NET (network stack), IPC (inter-process communication), USER (users) and UTS (hostname and domain name).*

Step 2 – Run multiple containers

Docker Hub (https://hub.docker.com) is the public registry of Docker images. Prefer **Official Images** and **Verified Publisher** images; unverified images may contain vulnerabilities or malicious software.

1. Run an **Nginx** web server container in the background (`--detach`), publishing container port 80 on host port 8080, with the name `nginx`:

```
docker container run --detach --publish 8080:80 --name nginx nginx
```

2. Access it with `curl localhost:8080` or open `http://localhost:8080` in a browser. The Welcome to nginx! page is displayed.
3. Run a **MongoDB** server using a specific version tag (4.4) instead of `latest`. Host port 8081 is used because 8080 is already taken:

```
docker container run --detach --publish 8081:27017 --name mongo mongo:4.4
```

4. Access it with `curl localhost:8081`. MongoDB replies: It looks like you are trying to access MongoDB over HTTP on the native driver port.
5. Check the running containers:

```
docker container ls
```

Containers are self-contained and isolated, so applications with different dependencies (for example Java 7 and Java 8) or several Nginx servers can run on the same host without conflict. Nothing except Docker had to be installed on the host.

Step 3 – Clean up

1. Stop the containers (names or the first three digits of the IDs can be used):

```
docker container stop nginx mongo
```

2. Remove the stopped containers. `docker system prune` removes all stopped containers, unused networks, dangling images and build cache, so use it with care:

Terminal

```
$ docker container run --detach --publish 8080:80 --name nginx nginx
5e1bf0e6b926bd73a66f98b3cbe23d04189c16a43d55dd46b8486359f6fdf048
$ docker container run --detach --publish 8081:27017 --name mongo mongo:4.4
d6777df89fea6f0b8c4e2a1d3b57c9e04f81a2b6c3d5e7f9012345678abcdef0
$ docker container ls
CONTAINER ID   IMAGE       COMMAND                  STATUS         PORTS                      NAMES
d6777df89fea   mongo:4.4   "docker-entrypoint.s…"   Up 5 seconds   0.0.0.0:8081->27017/tcp    mongo
5e1bf0e6b926   nginx       "nginx -g 'daemon of…"   Up 1 minute    0.0.0.0:8080->80/tcp       nginx
```

Fig 8.5 – Running containers

Terminal

```
$ docker container stop nginx mongo
nginx
mongo
$ docker container rm nginx mongo
nginx
mongo
$ docker system prune
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - all dangling build cache
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

Fig 8.6 – Stopping and removing the containers

```
docker container rm nginx mongo
docker system prune
```

APPLICATIONS:

Containers package an application with its dependencies and start in seconds. They are used for microservices, continuous integration and delivery, and for running many isolated applications on a single host to use resources efficiently.

RESULT:

Thus the first container was created and executed using Docker successfully.

OUTPUT:

Terminal

```
$ docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest

$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image which runs
    the executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which
    sent it to your terminal.
```

Fig 9.1 – docker pull and docker run hello-world

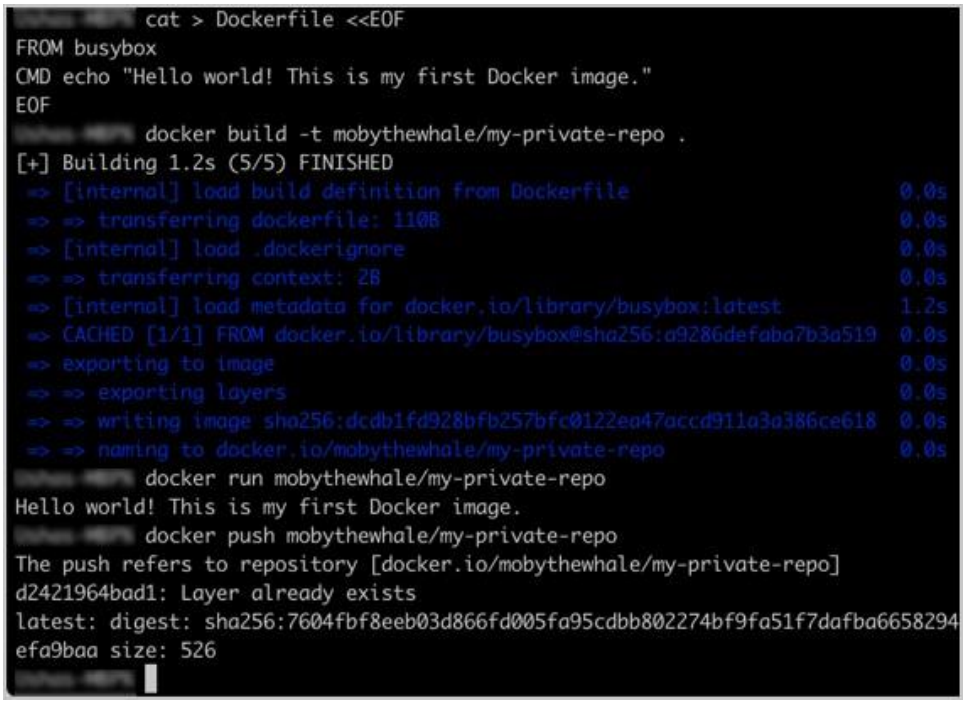


Fig 9.2 – docker build, run and push

EX NO. : 9 DATE :	Run a Container from Docker Hub

AIM:

To find a procedure to run a container from Docker Hub.

REQUIREMENTS:

- Docker Engine or Docker Desktop (see Experiment 8) and a free Docker ID.

PROCEDURE:

Step 1 – Sign up for a Docker account

Create a Docker ID at https://hub.docker.com. A Docker ID gives access to Docker Hub repositories, lets you explore images from the community and verified publishers, and is needed to share images.

Step 2 – Create your first repository

1. Sign in to Docker Hub.
2. Select **Create a Repository** on the welcome page.
3. Name it `<your-username>/my-private-repo` and set the visibility to **Private**.
4. Select **Create**.

Step 3 – Install Docker and sign in

1. Install Docker Desktop (Windows) or Docker Engine (Ubuntu VM, see Experiment 8).
2. Sign in from the command line (or from Docker Desktop) using your Docker ID:

```
docker login
```

Step 4 – Pull and run a container image from Docker Hub

1. Search for an image and pull the `hello-world` image from Docker Hub:

```
docker search hello-world
docker pull hello-world
docker images
```

2. Run the image locally:

```
docker run hello-world
```

3. Try something more ambitious – an interactive Ubuntu container – and check the containers afterwards:

```
docker run -it ubuntu bash
exit
docker ps -a
```

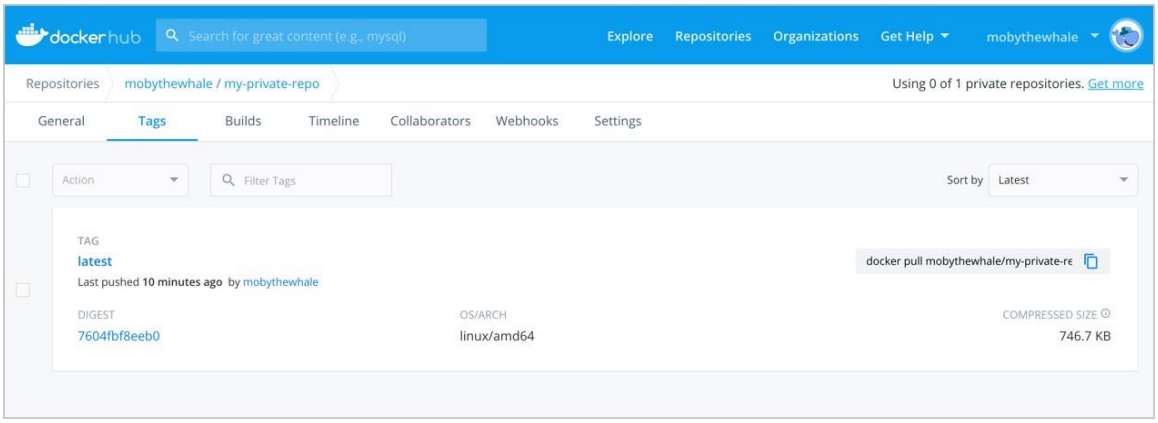


Fig 9.3 – Repository tags on Docker Hub

Step 5 – Build and push a container image to Docker Hub

1. Create a file named `Dockerfile` that specifies the application:

```
# syntax=docker/dockerfile:1
FROM busybox
CMD echo "Hello world! This is my first Docker image."
```

2. Build the image (do not forget the dot at the end) and test it locally:

```
docker build -t <your_username>/my-private-repo .
docker run <your_username>/my-private-repo
```

3. Push the image to Docker Hub:

```
docker push <your_username>/my-private-repo
```

Note: *You must be signed in to Docker Hub (docker login) and the image must be named <your_username>/<repository>, exactly as the repository on Docker Hub.*

4. Open the repository on Docker Hub. It now shows a new **latest** tag under **Tags**.

You have successfully signed up for a Docker account, created a repository, pulled and run an existing image from Docker Hub, built your own image and pushed it to Docker Hub.

APPLICATIONS:

Docker Hub acts as a central library of ready-made images. Teams pull base images, share their own images and deploy the same container unchanged on a laptop, a VM or a cloud platform.

RESULT:

Thus a container was run from Docker Hub, and a custom image was built and pushed to Docker Hub successfully.